Blockchain Technology

Group C Laboratory Practicals: BC-1 to BC-5

Detailed explanation with theory, requirements, step-by-step procedure, code, expected output and viva questions

Practical	Title
BC-1	Install Metamask and study spending Ether per transaction
BC-2	Create your own wallet using Metamask for crypto transactions
BC-3	Smart contract on a test network for a customer's bank account (deposit, withdraw, show balance)
BC-4	Survey report on types of blockchains and their real-time use cases
BC-5	Program to create a Business Network using Hyperledger

How to use this document: each practical is written in the usual lab-journal order (Aim, Objectives, Theory, Requirements, Procedure, Code/Output, Conclusion, Viva). Replace the sample addresses and hashes with your own screenshots and values when you submit. Always use a **test network** (Sepolia) and never use your real-money wallet or recovery phrase for lab work.

BC-1: Install Metamask and Study Spending of Ether per Transaction

Aim

To install the Metamask wallet, connect it to an Ethereum test network, and study how much Ether is spent (gas fee) in each transaction.

Objectives

- Understand what a crypto wallet and Metamask are.
- Learn the terms gas, gas limit, gas price, base fee and priority fee.
- Calculate the transaction fee and compare costs at different speeds.

Theory

Metamask is a browser extension (and mobile app) that works as a non-custodial Ethereum wallet and a gateway to decentralised applications (dApps). It stores your private keys locally, signs transactions and talks to the blockchain through a node provider (for example Infura). Whoever knows the 12-word Secret Recovery Phrase controls the wallet.

Every transaction that changes the state of Ethereum needs computational work from validators. The user pays for it in **gas**, paid in Ether.

Term	Meaning
Gas	Unit measuring the computational effort. A simple ETH transfer uses a fixed 21,000 gas.
Gas limit	Maximum gas the sender is willing to use. Unused gas is refunded.
Gwei	Unit of gas price. 1 gwei = 10^-9 ETH (one billionth of an ETH).
Base fee	Minimum price per gas set by the network (EIP-1559). It is burned.
Priority fee (tip)	Extra price per gas paid to the validator to include the transaction faster.
Max fee	Highest total price per gas you allow (base fee + tip).

Formula: Transaction fee = Gas used x (Base fee + Priority fee). Example: 21,000 x (12 + 2) gwei = 294,000 gwei = 0.000294 ETH.

Requirements

- Computer with Chrome, Firefox, Brave or Edge, and internet access
- Metamask extension (from metamask.io only, to avoid fake copies)
- Sepolia test ETH from a faucet
- Block explorer: sepolia.etherscan.io

Procedure

1. Open the official site metamask.io and click **Download**; add the extension to your browser and pin it.
2. Click **Create a new wallet**, agree to the terms and set a strong password.
3. Write the 12-word Secret Recovery Phrase on paper, confirm it when asked and keep it offline. Never share or screenshot it.
4. Open the network drop-down at the top and enable **Show test networks** (Settings > Advanced). Select **Sepolia**.

5. Copy your account address (starts with 0x) and request free test ETH from a Sepolia faucet (for example Google Cloud Web3 faucet or Alchemy faucet). Wait until the balance appears.
6. Click **Send**, paste a second address (or another of your own accounts), enter 0.001 ETH.
7. Before confirming, click the gas/Edit option. Note the estimated **gas fee**, **gas limit**, **max base fee** and **priority fee**.
8. Repeat the transaction with the three speed options (Low / Market / Aggressive) and record the fee for each.
9. After confirmation open **Activity > the transaction > View on block explorer**. Record the transaction hash, block number, gas used, gas price and transaction fee.
10. Compare the fee for each speed and calculate: fee = gas used x gas price.

Observation Table (fill with your values)

Sr	Amount sent	Speed	Gas limit	Gas used	Gas price (gwei)	Fee (ETH)
1	0.001	Low	21000	21000	__	__
2	0.001	Market	21000	21000	__	__
3	0.001	Aggressive	21000	21000	__	__

Conclusion

Metamask was installed and connected to the Sepolia network. A simple transfer costs 21,000 gas, and the actual Ether spent depends on the gas price, which changes with network congestion and the speed selected. Interacting with smart contracts uses more gas than a plain transfer.

Viva Questions

- What is gas and why is it needed?
- Difference between gas limit and gas used?
- What changed with EIP-1559?
- Why do we use a test network? Is test ETH valuable?
- Why must the recovery phrase be kept private?

BC-2: Create Your Own Wallet Using Metamask for Crypto Transactions

Aim

To create and manage a personal wallet using Metamask, create multiple accounts and carry out crypto transactions between them.

Theory

A blockchain wallet does not hold coins. The coins exist on the blockchain; the wallet holds the **keys** that prove ownership.

- **Private key:** secret number used to sign transactions. Never share it.
- **Public key and address:** derived from the private key; the address (0x...) is what others use to send funds.
- **Seed phrase (mnemonic):** 12 words from which all your accounts and keys are generated (HD wallet, BIP-39). Anyone with it can restore your wallet.
- **Hot vs cold wallet:** Metamask is a hot wallet (connected to internet). Hardware wallets are cold storage.
- **Digital signature:** every transaction is signed with the private key (ECDSA) and verified by nodes using the public key.

Requirements

- Metamask installed (BC-1)
- Sepolia test ETH
- Etherscan (Sepolia) for verification

Procedure

1. Open Metamask and confirm you are on the Sepolia network.
2. Rename **Account 1** to something meaningful (for example Wallet-A) from the account options.
3. Click the account icon > **Add account** and create **Account 2** (Wallet-B). Both accounts come from the same seed phrase.
4. Copy the address of Wallet-B (click the address to copy). Verify the first and last 4 characters before pasting.
5. From Wallet-A click **Send**, paste the Wallet-B address, enter an amount (example 0.01 ETH), review the gas fee and click **Confirm**.
6. Wait for the status to change from Pending to **Confirmed** in the Activity tab.
7. Open the transaction on Etherscan and note: Transaction Hash, Status, Block, From, To, Value, Transaction Fee.
8. Switch to Wallet-B and verify the received balance. Optionally send a part of it back to Wallet-A.
9. (Optional) Import an account using a private key or use **Account details > Show private key** to understand key export, then delete the imported test account.
10. (Optional) Click **Receive** to show the QR code of your address.

Sample Transaction Record

Field	Value (example, replace with yours)
From	0xAbc...1234 (Wallet-A)
To	0xDef...5678 (Wallet-B)
Value	0.01 SepoliaETH
Txn hash	0x9f3c...e21a
Status	Success, 1+ confirmations
Txn fee	0.000xx ETH

Security rules: store the seed phrase offline; beware of fake Metamask sites and support scams; Metamask support never asks for your phrase; always double-check the address before sending, because blockchain transactions cannot be reversed.

Conclusion

A self-custodial wallet with two accounts was created and Ether was transferred between them. The transaction was signed with the private key and recorded permanently on the Sepolia blockchain, which can be verified on Etherscan.

Viva Questions

- What is the difference between a wallet and an account?
- What is a seed phrase and how is it used?
- Why are blockchain transactions irreversible?
- Difference between custodial and non-custodial wallets?
- How is a transaction signed and verified?

BC-3: Smart Contract for a Customer's Bank Account

Aim

To write, deploy and test a Solidity smart contract on a test network that allows a customer to deposit money, withdraw money and show the balance.

Theory

A **smart contract** is a program stored on the blockchain that runs automatically when its functions are called. Once deployed its code cannot be changed, and every execution is verified by all nodes. **Solidity** is the main language for Ethereum smart contracts. The **Ethereum Virtual Machine (EVM)** executes the compiled bytecode. **Remix IDE** is a browser-based IDE to write, compile, deploy and debug contracts.

Concept used	Role in this program
address / msg.sender	Account that calls the function (the customer).
msg.value	Amount of Wei sent along with a payable function call.
mapping(address => uint256)	Stores the balance of every customer.
payable	Allows a function to receive Ether.
require()	Checks a condition; reverts the transaction with a message if false.
event / emit	Writes a log on chain that front-ends and Etherscan can read.
view	Read-only function; costs no gas when called externally.

Requirements

- Remix IDE (remix.ethereum.org)
- Metamask with Sepolia test ETH
- Solidity compiler 0.8.x

Program

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {
    address public owner;                          // account that deployed the contract
    mapping(address => uint256) private balances;  // balance of each customer (in wei)

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);

    constructor() {
        owner = msg.sender;
    }

    // 1. Deposit money: send ETH along with the call
    function deposit() public payable {
        require(msg.value > 0, "Deposit must be greater than 0");
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    // 2. Withdraw money
    function withdraw(uint256 amount) public {
        require(amount > 0, "Amount must be greater than 0");
        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;                    // update state first
```

```
        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "Transfer failed");                        // then send the money
        emit Withdrawn(msg.sender, amount);
    }

    // 3. Show balance of the caller
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }

    // Extra: total ETH held by the contract
    function contractBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Code Explanation

- **Line 1-2:** license identifier and compiler version. ^0.8.20 means 0.8.20 or any compatible newer 0.8.x.
- **balances mapping:** works like a dictionary from customer address to amount. It is private, but note that blockchain data is never truly secret.
- **deposit():** payable, so it can accept Ether. msg.value is added to the caller's balance.
- **withdraw():** first validates the input, then reduces the balance, and only then sends Ether. This ordering is the *checks-effects-interactions* pattern and protects against reentrancy attacks.
- **showBalance():** view function that returns the caller's balance in wei (1 ETH = 10^18 wei).
- **Events:** provide an audit trail of deposits and withdrawals.

Procedure (Remix + Metamask + Sepolia)

1. Open remix.ethereum.org, create the file **BankAccount.sol** in the contracts folder and paste the program.
2. Go to the **Solidity Compiler** tab, choose version 0.8.20 (or higher 0.8.x) and click **Compile**. A green tick means success.
3. Open **Deploy and Run Transactions**. Set Environment to **Injected Provider - MetaMask**. Metamask asks to connect; select your account on Sepolia.
4. Select the contract BankAccount and click **Deploy**. Confirm the transaction in Metamask. Copy the deployed contract address and view it on Sepolia Etherscan.
5. **Deposit:** in the Value box enter 0.01 and choose *Ether* as the unit, then click the orange **deposit** button and confirm in Metamask.
6. **Show balance:** click **showBalance**. Expected output: 10000000000000000 (that is 0.01 ETH in wei).
7. **Withdraw:** enter 5000000000000000 (0.005 ETH in wei) in the withdraw box, click it and confirm.
8. Click **showBalance** again. Expected output: 5000000000000000.
9. Test failure cases: withdraw more than the balance (expect revert 'Insufficient balance') and deposit with value 0 (expect 'Deposit must be greater than 0').
10. Check the contract creation and each function call on Etherscan (Transactions and Events/Logs tabs) and take screenshots.

Expected Output

Operation	Input	Result
deposit()	Value = 0.01 ETH	Success, event Deposited emitted
showBalance()	-	10000000000000000
withdraw(5000000000000000)	0.005 ETH in wei	Success, event Withdrawn emitted

Operation	Input	Result
showBalance()	-	5000000000000000
withdraw(9000000000000000)	More than balance	Reverts: Insufficient balance

Common Errors

- **Compiler version mismatch:** select a compiler that satisfies the pragma line.
- **Out of gas / insufficient funds:** get more Sepolia ETH from the faucet.
- **Deposit fails with value 0:** make sure the Value field is greater than zero and the unit is correct (wei vs ether).
- **Remix VM instead of Metamask:** if the environment is a Remix VM, the contract is not on Sepolia.

Conclusion

A BankAccount smart contract with deposit, withdraw and show-balance operations was written in Solidity, deployed on the Sepolia test network through Remix and Metamask and tested successfully. Validation with require() and the checks-effects-interactions pattern keep the account safe.

Viva Questions

- What is a smart contract and can it be modified after deployment?
- What is the use of the payable keyword?
- What is msg.sender and msg.value?
- Why is the balance updated before sending the Ether?
- Difference between view and pure functions?
- What are events and why use them?

BC-4: Survey Report on Types of Blockchains and Real-Time Use Cases

Aim

To study the types of blockchains (public, private, consortium and hybrid) and survey their real-world applications.

Theory

A blockchain is a distributed, append-only ledger in which blocks of transactions are linked by cryptographic hashes and agreed upon by nodes through a consensus mechanism. Blockchains are classified by **who may read, write and validate** the data.

Comparison of Blockchain Types

Feature	Public	Private	Consortium	Hybrid
Access	Anyone can join	Single organisation controls access	Group of pre-selected organisations	Mix of public and private parts
Validators	Anonymous nodes	Authorised nodes of one owner	Authorised nodes of the members	Chosen per use case
Consensus	PoW, PoS	PBFT, Raft, PoA	PBFT, Raft, PoA	Varies
Speed	Slower	Fast	Fast	Fast to moderate
Decentralisation	High	Low	Medium	Medium
Transparency	Full	Restricted	Shared among members	Selective
Examples	Bitcoin, Ethereum, Solana	Hyperledger Fabric (single org), Multichain	Corda, Quorum, Hyperledger Fabric	XDC Network, Dragonchain

Advantages and Limitations

Type	Advantages	Limitations
Public	Open, censorship resistant, high trust and security	Low throughput, high energy/fees (PoW), no privacy
Private	High speed, privacy, easy governance	Centralised, less trust, single point of control
Consortium	Shared governance, privacy, efficient	Needs agreement among members, partial centralisation
Hybrid	Flexible, controlled data exposure	Complex design and integration

Real-Time Use Cases

Type	Use case	Example
Public	Cryptocurrency and payments; DeFi lending/trading; NFTs and digital ownership; DAO governance	Bitcoin, Ethereum, Uniswap, Aave
Private	Internal audit trails, enterprise asset tracking, document management	Company-owned Fabric networks, Multichain pilots

Type	Use case	Example
Consortium	Supply chain traceability, trade finance, interbank settlement, shared KYC	Walmart Food Trust (Fabric), R3 Corda in banking, JPMorgan Quorum
Hybrid	Healthcare records with public verification, real estate title registries, government records	XDC Network trade finance, Dragonchain use cases

Other sector applications: education certificate verification, e-voting pilots, land records, pharmaceutical drug traceability, insurance claims automation, digital identity, and energy trading. Check the current status of any named project before quoting it, because some platforms (for example TradeLens) have been discontinued or changed.

Suggested Structure of the Report

1. Title page and abstract
2. Introduction to blockchain (ledger, block, hash, consensus)
3. Classification of blockchains with the comparison table
4. Detailed study of each type with architecture diagram
5. Real-time use cases per type and per industry
6. Challenges: scalability, privacy, regulation, interoperability, energy
7. Conclusion and future scope
8. References (research papers, official documentation, websites)

Conclusion

The choice of blockchain depends on the need for openness, performance and privacy. Public chains suit open financial systems, private and consortium chains suit enterprises needing privacy and throughput, and hybrid chains balance both.

Viva Questions

- Difference between public and private blockchain?
- What is a permissioned blockchain?
- Which consensus algorithms suit consortium networks?
- Give one real use case for each type.
- What is the blockchain trilemma?

BC-5: Create a Business Network Using Hyperledger

Aim

To create a Hyperledger Fabric business network with multiple organisations, deploy a chaincode (smart contract) and perform transactions on the ledger.

Theory

Hyperledger is an open-source umbrella project of the Linux Foundation for enterprise blockchains. **Hyperledger Fabric** is a permissioned, modular platform in which known organisations form a business network.

Component	Description
Organisation	A member of the network (for example a bank, manufacturer or retailer).
Peer	Node that hosts the ledger and runs chaincode. Endorsing peers simulate and sign transactions.
Orderer	Orders transactions into blocks and delivers them to peers (Raft consensus).
Channel	Private sub-network with its own ledger shared only by selected organisations.
Chaincode	Smart contract written in Go, JavaScript or Java that holds the business logic.
MSP / CA	Membership Service Provider and Certificate Authority give each participant an identity.
World state and ledger	World state holds latest values (key-value DB); the ledger holds the immutable transaction history.

Transaction flow: client proposes a transaction > endorsing peers simulate and sign > client sends endorsed transaction to the orderer > orderer creates a block > peers validate and commit the block > ledger and world state are updated.

Important: Hyperledger Composer, which used .cto, .acl and .js files and the 'composer network' commands, is deprecated and no longer maintained. Current practice is to build the business network with Fabric's **test-network** and chaincode. If your syllabus or teacher explicitly asks for Composer, check with them; the Fabric method below is the working option on current systems.

Requirements

- Ubuntu 20.04/22.04 (or WSL2 on Windows, or a Linux VM), at least 4 GB RAM
- Docker and Docker Compose
- Git, curl, jq
- Go (1.20+), Node.js (18 LTS) and npm
- Hyperledger Fabric binaries and docker images with fabric-samples (a recent 2.5.x release)

Procedure

Step 1: Install prerequisites

```
sudo apt update
sudo apt install -y git curl jq docker.io docker-compose-v2
sudo usermod -aG docker $USER        # log out and log in again
docker --version && git --version
```

Step 2: Download Fabric samples, binaries and images

```
mkdir -p ~/fabric && cd ~/fabric
```

```
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
cd fabric-samples/test-network
```

Step 3: Start the network and create a channel

This creates a business network with two peer organisations (Org1, Org2) and one ordering service, plus the channel **mychannel**.

```
./network.sh down                     # clean any old network
./network.sh up createChannel -c mychannel -ca
docker ps                             # verify peer, orderer and CA containers
```

Step 4: Write the chaincode (business logic)

A simple asset-tracking contract in JavaScript. Save as **fabric-samples/my-chaincode/lib/assetContract.js**. Also add a **package.json** with the dependencies fabric-contract-api and fabric-shim, and an **index.js** that exports the contract.

```
'use strict';
const { Contract } = require('fabric-contract-api');

class AssetContract extends Contract {

    async InitLedger(ctx) {
        const assets = [
            { ID: 'asset1', Owner: 'Org1-Manufacturer', Product: 'Laptop', Value: 500 },
            { ID: 'asset2', Owner: 'Org2-Retailer',     Product: 'Phone',  Value: 300 },
        ];
        for (const a of assets) {
            await ctx.stub.putState(a.ID, Buffer.from(JSON.stringify(a)));
        }
    }

    async CreateAsset(ctx, id, owner, product, value) {
        const exists = await this.AssetExists(ctx, id);
        if (exists) throw new Error(`Asset ${id} already exists`);
        const asset = { ID: id, Owner: owner, Product: product, Value: parseInt(value) };
        await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
        return JSON.stringify(asset);
    }

    async ReadAsset(ctx, id) {
        const data = await ctx.stub.getState(id);
        if (!data || data.length === 0) throw new Error(`Asset ${id} does not exist`);
        return data.toString();
    }

    async TransferAsset(ctx, id, newOwner) {
        const asset = JSON.parse(await this.ReadAsset(ctx, id));
        asset.Owner = newOwner;
        await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
        return JSON.stringify(asset);
    }

    async AssetExists(ctx, id) {
        const data = await ctx.stub.getState(id);
        return data && data.length > 0;
    }

    async GetAllAssets(ctx) {
        const results = [];
        const it = await ctx.stub.getStateByRange('', '');
        let r = await it.next();
        while (!r.done) {
            results.push(JSON.parse(r.value.value.toString()));
            r = await it.next();
        }
        return JSON.stringify(results);
    }
}
module.exports = AssetContract;
```

```
// index.js
'use strict';
const AssetContract = require('./lib/assetContract');
module.exports.AssetContract = AssetContract;
module.exports.contracts = [AssetContract];
```

Alternatively, skip writing it and use the ready-made sample at **fabric-samples/asset-transfer-basic/chaincode-javascript**, which has the same functions.

Step 5: Deploy the chaincode on the channel

```
./network.sh deployCC -ccn assetcc -ccp ../my-chaincode -ccl javascript
# for the ready-made sample:
# ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript
```

Step 6: Set environment variables for Org1 and invoke transactions

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
ORGS=${PWD}/organizations
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_ADDRESS=localhost:7051
export CORE_PEER_TLS_ROOTCERT_FILE=$ORGS/peerOrganizations/org1.example.com/peers/\
peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$ORGS/peerOrganizations/org1.example.com/users/\
Admin@org1.example.com/msp
```

Note: the lines ending with a backslash continue on the next line; type each as a single line in the terminal. Now store the certificate paths in short variables:

```
ORDERER_CA=$ORGS/ordererOrganizations/example.com/orderers/orderer.example.com/msp/\
tlscacerts/tlsca.example.com-cert.pem
ORG1_TLS=$ORGS/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
ORG2_TLS=$ORGS/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

Initialise the ledger (this invoke command is reused for every write transaction, only the -c part changes):

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com \
  --tls --cafile $ORDERER_CA -C mychannel -n assetcc \
  --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS \
  --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS \
  -c '{"function":"InitLedger","Args":[]}'
```

Query the ledger (read-only, no ordering needed):

```
peer chaincode query -C mychannel -n assetcc -c '{"Args":["GetAllAssets"]}'
peer chaincode query -C mychannel -n assetcc -c '{"Args":["ReadAsset","asset1"]}'
```

Create and transfer an asset using the same long `peer chaincode invoke` command with these -c arguments:

```
-c '{"function":"CreateAsset","Args":["asset3","Org1-Manufacturer","Tablet","250"]}'
-c '{"function":"TransferAsset","Args":["asset3","Org2-Retailer"]}'
```

Step 7: Verify and shut down

```
peer chaincode query -C mychannel -n assetcc -c '{"Args":["ReadAsset","asset3"]}'
# expected: {"ID":"asset3","Owner":"Org2-Retailer","Product":"Tablet","Value":250}

./network.sh down                     # stops containers and removes the network
```

Expected Output

- docker ps shows containers: peer0.org1, peer0.org2, orderer, and the chaincode containers.
- InitLedger returns status:200 (invoke successful).
- GetAllAssets returns the JSON array of assets; ReadAsset returns one asset.
- After TransferAsset, the Owner field changes to Org2-Retailer, showing ownership transfer between organisations on the shared ledger.

Troubleshooting

- **permission denied on docker:** add your user to the docker group and re-login.
- **Port already in use:** run ./network.sh down and docker system prune.
- **peer: command not found:** make sure the bin folder is on PATH as shown above.
- **Chaincode build failure:** check package.json dependencies and the Node version.
- **Script URL or version differs:** follow the latest instructions at hyperledger-fabric.readthedocs.io (Using the Fabric test network).

Conclusion

A Hyperledger Fabric business network with two organisations and an ordering service was created, a channel was formed, chaincode was deployed and asset transactions were invoked and queried. This shows how permissioned blockchains provide shared, tamper-proof records between business partners.

Viva Questions

- How does Hyperledger Fabric differ from Ethereum?
- What are the roles of peers, orderers and channels?
- What is chaincode?
- Explain the execute-order-validate transaction flow.
- What is the world state?
- Why is Hyperledger Composer no longer used?